

ASSIGNMENT 18.5

Name:G.Charanya

HT No:2303A510G6

Batch:23

Task 1 – IP Address Location Tracker API

Task:

Find geographical location details of an IP address.

Instructions:

- Accept IP input from user.
- Handle invalid IP format and API failures.
- Display city, country, ISP, and region.

Expected Output:

- A script that tracks IP location with proper exception handling.

```
1  '''use IP Address Location Tracker API
2  design geographical location details of an IP address'''
3  import requests
4  def get_ip_location(ip_address):
5      url = f"http://ip-api.com/json/{ip_address}"
6      response = requests.get(url)
7      if response.status_code == 200:
8          data = response.json()
9          if data['status'] == 'success':
10             return {
11                 'ip': ip_address,
12                 'country': data.get('country', 'N/A'),
13                 'region': data.get('regionName', 'N/A'),
14                 'city': data.get('city', 'N/A'),
15                 'latitude': data.get('lat', 'N/A'),
16                 'longitude': data.get('lon', 'N/A'),
17                 'isp': data.get('isp', 'N/A')
18             }
19         else:
20             return {'error': f"Failed to retrieve location for IP: {ip_address}"}
21     else:
22         return {'error': f"HTTP error {response.status_code} for IP: {ip_address}"}
23 if __name__ == "__main__":
24     ip = input("Enter an IP address: ")
25     location_info = get_ip_location(ip)
26     print(location_info)
```

OUTPUT:

```
Enter an IP address:
{'ip': '', 'country': 'India', 'region': 'Telangana', 'city': 'Warangal', 'latitude': 17.9958, 'longitude': 79.57,
82, 'isp': 'RailTel Corporation Of India Ltd'}
PS C:\Users\ADMIN>
```

Task 2 – YouTube Channel Statistics API

Task:

Fetch YouTube channel details using Google API.

Instructions:

- Input channel ID or name.
- Handle quota errors, invalid ID, request failures.
- Display subscribers, views, and video count.

Expected Output:

- A working YouTube stats fetcher with error handling.

```
1  '''design Fetch YouTube channel details
2  using YouTube Channel Statistics API and google API '''
3  import requests
4  import json
5  def fetch_channel_details(api_key, channel_id):
6      url = f'https://www.googleapis.com/youtube/v3/channels?part=snippet,statistics&id={channel_id}&ke
7      response = requests.get(url)
8      if response.status_code == 200:
9          data = response.json()
10         if 'items' in data and len(data['items']) > 0:
11             channel_info = data['items'][0]
12             channel_details = {
13                 'title': channel_info['snippet']['title'],
14                 'description': channel_info['snippet']['description'],
15                 'subscriber_count': channel_info['statistics']['subscriberCount'],
16                 'video_count': channel_info['statistics']['videoCount'],
17                 'view_count': channel_info['statistics']['viewCount']
18             }
19             return channel_details
20         else:
21             print('Channel not found.')
22             return None
23     else:
24         print(f'Error fetching data: {response.status_code}')
25         return None
```

OUTPUT:

```
AC Project/Q218.5.py"
Enter YouTube Channel ID: https://www.youtube.com/@saregamasout
Error fetching data: 400
```

Task 3– Public Transport Live Status API

Task:

Connect to a Transport API to check live bus/train status.

Instructions:

- Input route or train number.
- Handle invalid input, unavailable service, timeout issues.
- Display arrival/departure updates.

Expected Output:

- A real-time transport tracker with retry mechanism.

```

1  '''design Public Transport Live Status API
2  use Transport API to check live bus/train status'''
3  import requests
4  API_KEY = "YOUR_API_KEY" # Replace with your Transport API key
5  def get_transit_details(source, destination):
6      url = "https://maps.googleapis.com/maps/api/directions/json"
7      params = {
8          "origin": source,
9          "destination": destination,
10         "mode": "transit",
11         "key": API_KEY
12     }
13     response = requests.get(url, params=params)
14     data = response.json()
15     if data["status"] == "OK":
16         route = data["routes"][0]["legs"][0]
17         print("From:", route["start_address"])
18         print("To:", route["end_address"])
19         print("Duration:", route["duration"]["text"])
20         print("\nSteps:")

```

OUTPUT:

```

AC Project/Q518.5.py
Enter source: chenni central
Enter destination: tambaram
Error: REQUEST_DENIED
PS C:\Users\ADMIN>

```

Task 4 – Online Quiz Questions API

Task:

Fetch quiz questions from an Online Trivia API.

Instructions:

- Request 5 multiple-choice questions.
- Handle API errors, invalid category selection.
- Display questions neatly.

Expected Output:

- A quiz generator script with error-safe API handling.

```

1  '''use Fetch quiz questions from an Online Trivia API.
2  design Online Quiz Questions API'''
3  import requests
4  import json
5  def fetch_online_quiz_questions(api_url):
6      response = requests.get(api_url)
7      if response.status_code == 200:
8          data = response.json()
9          return data['results'] # Assuming the API returns a 'results' field with quiz questions
10     else:
11         print(f'Error fetching data: {response.status_code}')
12         return None
13  if __name__ == "__main__":
14      api_url = 'https://opentdb.com/api.php?amount=10&type=multiple' # Example API URL for fetching q
15      quiz_questions = fetch_online_quiz_questions(api_url)
16      if quiz_questions:
17          for idx, question in enumerate(quiz_questions):
18              print(f"Question {idx + 1}: {question['question']}")
19              print(f"Options: {' '.join(question['incorrect_answers'] + [question['correct_answer']])}")
20              print(f"Correct Answer: {question['correct_answer']}\n")
21

```

OUTPUT:

```

Question 1: In "Sonic the Hedgehog 3" for the Sega Genesis, what is the color of the second Chaos Emerald you can get from Special Stages?
Options: Blue, Green, Magenta, Orange
Correct Answer: Orange

Question 2: In Magic: The Gathering, what card's flavor text is "Catch!"?
Options: Stone-Throwing Devils, Ember Shot, Throwing Knife, Lava Axe

```

Task 5– E-Commerce Product Price Comparison API

Task:

Integrate a Product Search API to fetch item pricing details.

Instructions:

- User enters product name.
- Handle missing results, invalid API key, slow responses.
- Display product name, price, and availability.

Expected Output:

- A product search tool with graceful failure handling.

```

1  '''design Integrate a Product Search API to fetch item pricing details.
2  use E-Commerce Product Price Comparison API'''
3  import requests
4  import json
5  def fetch_product_price(api_key, product_name):
6      url = "https://serpapi.com/search.json"
7      params = {
8          "engine": "google_shopping",
9          "q": product_name,
10         "api_key": api_key
11     }
12     response = requests.get(url, params=params)
13     data = response.json()
14     if "shopping_results" in data:
15         product = data["shopping_results"][0]
16         return {
17             "product_name": product.get("title"),
18             "price": product.get("price"),
19             "source": product.get("source")
20         }
21     else:
22         print("No product found")

```

OUTPUT:

```

Enter product name: iphone 15
No product found

```